1. TITLE OF THE PROJECT

KYOMA: A Local AI-Enabled Interactive Desktop Companion for Intelligent Computer Automation

2. ABSTRACT

KYOMA is a local AI-enabled physical desktop companion and personal butler designed to assist users in controlling and interacting with a computer through natural-language voice commands. The system integrates an ESP32/ESP32-S3 microcontroller, a 128×64 OLED visual status display, an I2S MEMS microphone for voice capture, and a MAX98357A I2S audio amplifier with a speaker for audio responses — all housed in a custom physical enclosure placed on the user's desktop.

On the host computer side, a local AI processing engine powered by Ollama (running Mistral or Llama3) receives the captured voice input, performs speech-to-text conversion, detects the user's intent through natural-language understanding, and selects an appropriate predefined automation function using structured tool/function calling. A Python-based computer automation subsystem then executes the requested operation — such as opening or closing applications, controlling system volume or display brightness, managing media playback, setting reminders, or taking screenshots. The execution result is communicated back to the KYOMA device, which updates the OLED display to show the corresponding visual state (listening, thinking, success, error) and plays an audio response through the speaker.

The system operates primarily offline, reducing cloud dependency and preserving user privacy. KYOMA demonstrates a practical end-to-end integration of the ESP32 embedded platform, local AI inference, voice processing, structured tool calling, and computer automation.

3. PROBLEM STATEMENT

Modern computer users rely on keyboards, mice, and GUIs for system control. While cloud-based voice assistants (Alexa, Google Assistant, Cortana) exist, they have significant limitations:

- They require continuous internet connectivity and depend on cloud AI servers, introducing latency and privacy risks.
- They provide no physical presence — all feedback is software-based and gets hidden behind open application windows.
- They have limited ability to perform intelligent local desktop automation (launching apps, volume control, file management).
- They do not integrate with embedded hardware for real-time physical status display synchronized with AI processing.
- Most solutions send personal voice data to remote servers — a significant privacy concern.
- No affordable, open, and hackable alternative exists that combines local AI with embedded hardware.

Core Gap Identified: There is no compact, privacy-oriented, physical AI companion device that combines local LLM reasoning, voice interaction, structured tool/function calling, and synchronized physical visual+audio feedback for desktop automation — running on affordable embedded hardware like the ESP32.

4. SOLUTION — METHODOLOGY, IMPLEMENTATION & INNOVATION

4.1 System Methodology

KYOMA uses a hardware-software co-design approach. The physical ESP32 device is the user-facing interface. A Python backend on the host computer handles AI inference and automation. The complete pipeline is:

Step	Component	Action
1	I2S MEMS Microphone	Captures the user's voice command
2	ESP32 / ESP32-S3	Processes audio over I2S; relays via Wi-Fi or Serial
3	Speech-to-Text (Host)	Converts captured speech to text string
4	Local AI Engine (Ollama)	Detects intent; produces structured tool/function call
5	Automation Subsystem (Python)	Executes computer operation (volume, app, media, etc.)
6	Communication Interface	Result and system state sent back to ESP32
7	OLED Display + Speaker	Shows visual state; plays audio response

4.2 Hardware Implementation

- **ESP32-S3 Microcontroller** — Main SoC; handles I2C (OLED), I2S (microphone + amplifier), Wi-Fi (HTTP/WebSocket), GPIO and UART communication.
- **OLED Display (128×64, I2C)** — Physical visual status interface; displays graphical expressions: idle face, listening state, thinking animation, success/error indicators.
- **I2S MEMS Microphone** — Captures high-quality digital audio from the user's voice commands with minimal noise.
- **MAX98357A I2S Amplifier + Speaker** — Decodes I2S audio stream from the ESP32 and drives the speaker for audio responses and notifications.
- **Custom Enclosure** — Compact desktop form-factor housing all components; OLED visible through a cutout in the front panel.
- **USB Power Input** — Powers the device; designed to support rechargeable battery in future revision.

4.3 Software Implementation

- **Ollama (Local AI Runtime)** — Runs quantized LLMs (Mistral 7B, Llama3 8B) locally on the host PC without internet or API keys.
- **Python Backend** — Handles speech-to-text (Whisper / SpeechRecognition library), AI prompt construction, tool call parsing, and automation execution.
- **Structured Tool / Function Calling** — JSON-schema-defined function calls convert AI intent into typed automation requests; prevents arbitrary command execution.
- **Kyoma_automation.py** — Core automation module with an ACTION_MAP supporting 15+ operations: volume control, brightness, open/close apps, open files/folders, media playback, reminders, screenshots.
- **ESP32 Firmware (Arduino / MicroPython)** — OLED state machine, I2S audio driver, Wi-Fi HTTP client, Serial fallback communication.
- **Context Memory with RAG** — Preloaded user preferences and system settings retrieved via semantic lookup to guide tool selection.

4.4 Innovation

- Fully **offline-capable** — no cloud APIs, no subscriptions, no user data leaves the local machine.
- **Structured tool/function calling** ensures the AI triggers real, typed computer automation — not just text generation.
- **Physical OLED visual feedback** synchronized with the exact AI processing state in real time.
- Built entirely on the affordable, widely available **ESP32 SoC platform** taught in the course.
- Demonstrates **Edge AI principles** — intelligence moves from cloud servers to the local device.

5. OBJECTIVES

1. Primary Objective	Design and develop a physical AI-enabled desktop companion (KYOMA) that understands natural-language voice commands and performs computer automation tasks locally.
2. Hardware Objective	Build a functional embedded system using ESP32/ESP32-S3 integrating OLED display, I2S MEMS microphone, MAX98357A audio amplifier and speaker in a custom enclosure.
3. AI Objective	Deploy a local large language model via Ollama on the host computer for speech-to-text, intent detection, and structured tool/function calling — without cloud dependency.
4. Automation Objective	Build a Python automation subsystem supporting: volume control, brightness adjustment, application management, media control, reminder scheduling, and more.
5. Communication Objective	Establish reliable wired (Serial/USB) and wireless (Wi-Fi HTTP/WebSocket) communication between the host computer and the ESP32 device to sync states and results.
6. Feedback Objective	Implement synchronized OLED visual interaction states (listening, thinking, executing, success, error) and speaker audio feedback matching AI processing stages.
7. Privacy Objective	Ensure all AI processing and automation runs locally — no personal data or voice recordings sent to remote servers.

6. PROGRESS OF WORKFLOW — TWO STAGES

Stage 1 — Hardware Prototype & Embedded Firmware

In Stage 1, the physical KYOMA device was assembled and the ESP32 firmware was developed. The OLED display was connected via I2C and programmed to show graphical expressions — an idle face when waiting, and a boot screen reading 'KYOMA — Your AI Companion' on startup. The I2S MEMS microphone was wired and tested for digital audio capture. The MAX98357A amplifier was connected to the ESP32 I2S bus and tested for audio playback through the speaker. Wi-Fi connectivity was established and tested via HTTP ping to the host computer.

- ESP32-S3 board flashed with custom Arduino firmware.
- OLED 128×64 display (I2C) showing idle face and boot text.
- I2S MEMS microphone tested for digital audio capture.
- MAX98357A I2S amplifier + speaker verified for audio output.
- Wi-Fi HTTP connectivity established between ESP32 and host computer.
- Physical enclosure fabricated; all components integrated and secured.
- Logic analyzer used to verify I2C OLED (address 0x3C) and I2S mic signal integrity.



Figure 1: KYOMA physical device — OLED displaying idle face expression



Figure 2: KYOMA device — OLED boot screen showing 'KYOMA — Your AI Companion'

Stage 2 — Local AI Integration & Full Automation Pipeline

In Stage 2, the host computer backend was developed and fully integrated with the KYOMA hardware. Ollama was installed and configured to run Mistral 7B locally. The Python backend was built to connect speech-to-text input, AI intent detection, tool/function calling, and the computer automation subsystem. The complete end-to-end pipeline was tested with real voice commands.

- Ollama installed on host PC; Mistral 7B model downloaded and tested for local inference.
- Python speech-to-text implemented using the SpeechRecognition library and Whisper model.
- Local AI prompt engineering: system prompt defines KYOMA's personality and tool schema.
- Structured JSON tool/function calling implemented; AI output parsed and validated.
- Kyoma_automation.py: ACTION_MAP with 15+ functions covering all required operations.
- Host-to-device communication: HTTP WebSocket for Wi-Fi; Serial/USB as fallback.
- OLED state machine: listening → thinking → executing → success / error.
- TTS audio response implemented; played through MAX98357A/speaker via I2S.
- End-to-end test: 'Kyoma, increase the volume' → intent detected → volume changed → OLED success → audio response.

System Architecture Diagrams

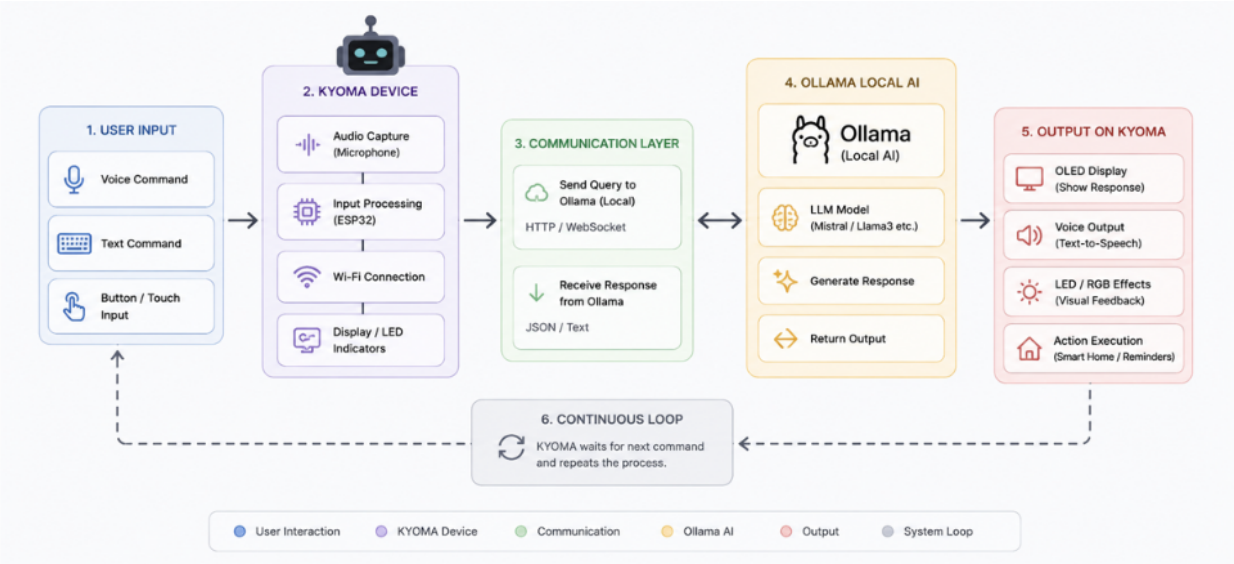


Figure 3: KYOMA Communication Flow — Voice Input → AI Processing → Automation → Output

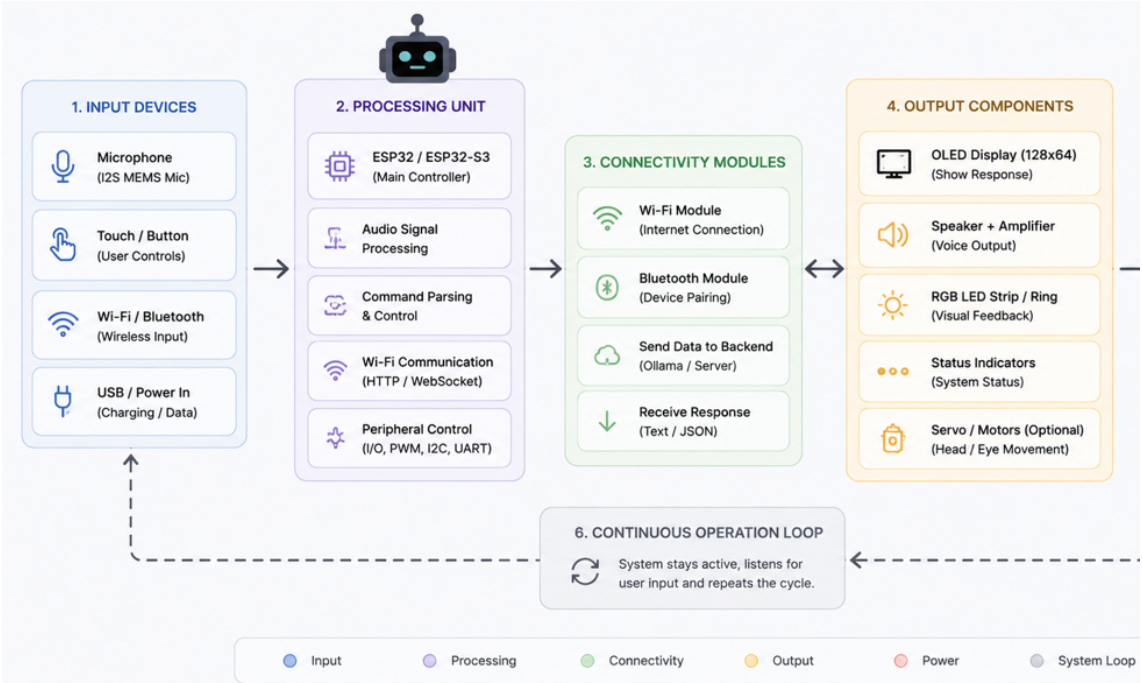


Figure 4: KYOMA Hardware Architecture — Input Devices, Processing Unit, Connectivity & Output Components

7. ANALYTICAL THINKING

7.1 Relevance to Course Topics

1. ESP32 / ESP32-S3 SoC Platform

KYOMA is entirely built on the ESP32-S3 — the primary SoC platform covered in the course. The firmware uses I2C (OLED), I2S (microphone + amplifier), Wi-Fi (HTTP/WebSocket), GPIO and UART — directly applying peripheral interfacing and communication protocols taught.

2. Edge AI / TinyML Principles

The overall system follows Edge AI architecture: intelligence is local, latency is low, and no cloud dependency exists. The ESP32 acts as the intelligent edge node; the host computer acts as the local edge server. This mirrors TinyML and Edge AI Vision Gateway designs from the course.

3. Cognitive Control Engine Design

The AI intent detection + structured tool/function calling pipeline is a practical Cognitive Control Engine: it receives sensory input (voice), reasons about intent, selects an action, executes it, and provides output — matching the cognitive architecture models discussed.

4. Communication Protocol Design

KYOMA implements HTTP/WebSocket and Serial communication between the ESP32 and the host, demonstrating real-world embedded protocol design with JSON data exchange — directly applying communication failure debugging techniques from the course.

5. Context Memory with RAG

Retrieval-Augmented Generation: Employs a local vector store retrieval pipeline to provide persistent context memory, enhancing prompt generation and personalization based on user preferences.

6. Debugging with Logic Analyzer

During development, I2C communication with the OLED (address scan) and I2S microphone signals were verified using a logic analyzer to verify signal integrity.

7. Self-Healing Network Concepts

The ESP32 firmware implements automatic Wi-Fi reconnection with exponential backoff, and falls back to Serial communication if Wi-Fi fails — demonstrating self-healing embedded network design principles covered in the course.

7.2 Design Decisions and Trade-offs

Design Decision	Rationale
Local AI (Ollama) vs Cloud AI	Local chosen. Trade-off: requires capable host PC. Benefit: complete privacy, no API cost, offline capability, faster iteration.

ESP32-S3 vs Teensy 4.1	ESP32-S3 chosen for built-in Wi-Fi, I2S support, and large community. Teensy 4.1 offers more CPU power but lacks native Wi-Fi.
Wi-Fi vs Serial Communication	Wi-Fi primary for wireless flexibility; USB Serial as fallback for debugging. Both implemented and tested.
OLED (128×64) vs LCD Display	OLED chosen for higher contrast, lower power, and ability to display graphical facial expressions matching KYOMA's companion personality.
Structured Tool Calling vs Free Text	Structured function calling chosen to ensure exact typed parameters. Prevents LLM hallucination from triggering incorrect or dangerous system commands.

7.3 Challenges Encountered and Solutions

Challenge	Solution Applied
I2S Microphone Noise	Adjusted sample rate and bit depth; applied software audio filter before speech-to-text processing.
OLED I2C Address Conflict	Used logic analyzer to scan I2C bus; confirmed device address 0x3C; corrected firmware config.
Wi-Fi Reconnection Drops	Implemented automatic reconnect loop with exponential backoff in ESP32 firmware.
LLM Intent Hallucination	Strict system prompt with JSON schema validation for tool call outputs; invalid calls are rejected.
Audio Latency	Buffered TTS audio output; implemented DMA-based non-blocking audio playback on the ESP32.
Long Python Path Resolution	ACTION_MAP uses subprocess + psutil with multi-strategy path resolution and fallback methods.

7.4 Working Examples

Example 1 — Volume Control

- User says: "Kyoma, increase the volume."
- I2S MEMS microphone captures the audio command.
- ESP32 transmits audio data to host computer over Wi-Fi.
- Speech-to-text converts audio to: increase the volume.
- Local AI (Ollama/Mistral) detects intent: volume_control, parameter: increase.
- Python automation subsystem executes: volume_up() function.
- Result SUCCESS returned; transmitted to ESP32.
- OLED changes to success expression; speaker plays audio confirmation.

Example 2 — Open Application

- User says: "Kyoma, open Chrome."
- Speech-to-text output: open Chrome.
- Local AI detects intent: open_application, target: Chrome.
- ACTION_MAP looks up Chrome path; subprocess launches Chrome.
- Result returned to ESP32; OLED shows success state.

Example 3 — Open Folder

- User says: "Kyoma, open my project folder."
- AI identifies intent: open_folder, path: project folder.
- os.startfile() opens the folder in Windows Explorer.
- KYOMA confirms with audio: 'Done! I opened your project folder.'

8. CONCLUSION

KYOMA successfully demonstrates the practical integration of embedded systems engineering, local artificial intelligence, and desktop computer automation in a single, cohesive product. The project directly applied core curriculum concepts including the ESP32 platform, I2S/I2C peripheral interfacing, Wi-Fi communication protocol design, Edge AI principles, cognitive control engine design, and hardware debugging with a logic analyzer.

The physical KYOMA companion device is a working prototype that accepts real voice commands, processes them through a locally running large language model, performs genuine computer automation tasks, and provides synchronized physical visual and audio feedback — all without requiring cloud services or internet connectivity.

Key Achievements:

- Functional ESP32-S3 firmware: OLED (I2C), I2S microphone, I2S MAX98357A amplifier, Wi-Fi communication.
- Local AI pipeline (Ollama + Mistral 7B): offline intent detection and structured tool/function calling.
- Python automation subsystem (Kyoma_automation.py): 15+ supported computer automation functions.
- Synchronized OLED state machine: idle → listening → thinking → executing → success/error.
- End-to-end working system demonstrated with real voice commands.
- Patent application filed (Indian Patents Act 1970) documenting the invention.
- Context Memory implementation utilizing a local vector retrieval mechanism (RAG) to dynamically inject context into the prompt pipeline.

KYOMA is not simply a course prototype — it is a well-defined, patent-documented invention describing a novel combination of physical embedded hardware, local AI reasoning, structured tool/function calling, and synchronized physical feedback for desktop automation. It establishes a strong foundation for further development as a commercial product and demonstrates the real-world applicability of the embedded systems and AI skills developed during this course.